# ANALISIS ALGORITME

## ANALISIS ALGORITME REKURSIF (*RECURRENCE*)

# CAPAIAN PEMBELAJARAN

- Mahasiswa dapat menganalisis kompleksitas tipe algoritme rekursif (recurrence).

# RECURSION / FUNGSI REKURSIF

- Teknik penyelesaian masalah yang solusinya bergantung pada masalah serupa dengan ukuran yang lebih kecil (subproblem).

Faktorial:

$$\text{fact}(n) = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot \text{fact}(n-1) & \text{if } n > 0 \end{cases}$$

Faktor persekutuan terbesar:

$$\gcd(x, y) = \begin{cases} x & \text{if } y = 0 \\ \gcd(y, \text{remainder}(x, y)) & \text{if } y > 0 \end{cases}$$

Jumlah langkah pada menara Hanoi:

$$\text{hanoi}(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2 \cdot \text{hanoi}(n-1) + 1 & \text{if } n > 1 \end{cases}$$

# RECURSION / FUNGSI REKURSIF

- Banyak permasalahan yang akan lebih mudah dituliskan menggunakan teknik recursion.
- Seluruh perulangan iteratif dapat dituliskan dalam bentuk rekursif. Sebaliknya belum tentu*.

$$A(m,n) = \begin{cases} n+1 & \text{if } m = 0 \\ A(m-1,1) & \text{if } m > 0 \text{ and } n = 0 \\ A(m-1, A(m,n-1)) & \text{if } m > 0 \text{ and } n > 0. \end{cases}$$

Fungsi Ackermann:

- Berapakah nilai dari A(4, 2)?

* Topik ini tidak akan dibahas, tapi yang tertarik bisa mengecek fungsi Ackermann di Wikipedia dan literatur lainnya.

# RECURSION / FUNGSI REKURSIF

## The Ackermann function

A(4, 2) =

```
2003529930406846464979072351560255750447825475569751419265016973710894059556311 4
5308950613088093334810103823434290726318182294938211881266886950636476154702916 5
0418719163515879663472194429309279820843091048559905701593189596395248633723672 0
3002916969592156108764948889254090805911457037675208500206671563702366126359747 1
4480711177481588091413574272096719015183628256061809145885269982614142503012339 1
1082736038437678764490432059603791244909057075603140350761625624760318637931264 8
4703743782954975613770981604614413308692118102485959152380195331030292162800160 5
6867010565164675056803874152946384224484529253736144253361437372908830379460127 4
7249584148649159306472520151556939226281806916507963810641322753072671439981585 0
8811292628901134237782705567421080070065283963322155077831214288551675554073345 1
0721311242739956298271976915005488390522380435704584819795639315785351001899200 0
0241419637068135598404640394721940160695176901561197269823378900176415171900511 3
3466306898140219383481435426387306539552969691388024158161859561100640362119796 1
```

http://bit.ly/2MYl7AZ

Tidak semudah itu Ferguso.

# STRUKTUR REKURSIF

- Langkah basis: Langkah yang atomik, tidak memanggil dirinya sendiri.
- Langkah rekursif: Langkah yang memanggil dirinya sendiri.

Langkah Basis

$$\text{fact}(n) = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot \text{fact}(n-1) & \text{if } n > 0 \end{cases}$$

Langkah Rekursif

# LATIHAN

- Buatlah algoritme rekursif dari permasalahan berikut:
  - Perkalian a x b hanya dengan menggunakan operasi penjumlahan.
  - Pemangkatan $a^b$, a dan b adalah bilangan bulat non-negatif, dengan menggunakan operasi perkalian.
  - Menghitung jumlah digit dari suatu bilangan bulat.
  - Mencari elemen terbesar pada array yang berisi n bilangan bulat tak negatif.

# JAWABAN

- Perkalian a x b hanya dengan menggunakan operasi penjumlahan.

```
int multiply(int a, int b){
   if (b == 0) return 0;
   else return a + multiply(a, b-1);
}
```

- Pemangkatan $a^b$, a dan b adalah bilangan bulat non-negatif, dengan menggunakan operasi perkalian.

```
int power(int a, int b){
   if (b == 0) return 1;
   else return a * power(a, b-1);
}
```

# JAWABAN

- Menghitung jumlah digit dari suatu bilangan bulat.

```
int hitungDigit(int n){
      if(n == 0) return 0;
      else return (n%10) + hitungDigit(n/10);
}
```

# JAWABAN

- Mencari elemen terbesar pada array yang berisi n bilangan bulat taknegatif.

```
// in main(): searchMax(numbers, -1, n);

int searchMax(int numbers[], int max, int n){
    if(n == 0) return max;
    else{
        if(numbers[n-1] > max) max = numbers[n-1];
        searchMax(numbers, max, n-1);
    }
}
```

# MENGANALISIS KOMPLEKSITAS SUATU ALGORITME REKURSIF

- Algoritme yang memiliki struktur rekursif tidak bisa dianalisis dengan cara yang sama seperti algoritme iteratif.
- Misalnya pada binary search berikut. Berapa kompleksitasnya?

```
1  int binarySearch(int arr[], int l, int r, int x)
2  {
3      if (r >= l) {
4          int mid = l + (r - l) / 2;
5          if (arr[mid] == x)
6              return mid;
7          if (arr[mid] > x)
8              return binarySearch(arr, l, mid - 1, x);
9          return binarySearch(arr, mid + 1, r, x);
10     }
11     return -1;
12 }
```

# MENGANALISIS KOMPLEKSITAS SUATU ALGORITME REKURSIF

- Algoritme rekursif dianalisis dengan menyelesaikan recurrence relation T(n) dari algoritme tersebut.

$$T(n) = aT(n/b) + f(n)$$

Jumlah subproblem.

Ukuran dari subproblem relatif terhadap ukuran masalah asli.

Waktu yang diperlukan untuk membagi subproblem dan menyatukannya kembali.

# CONTOH: *BINARY SEARCH*

- Pada setiap angkah rekursif, binary search membagi ukuran list masukan menjadi dua, dan melakukan proses di salah satu bagian saja.




- Karena ukuran subproblem adalah setengah dari ukuran aslinya, nilai b = 2.
- Walaupun list terbagi menjadi dua bagian, proses selanjutnya hanya dikerjakan pada salah satu bagian. Dengan demikian a = 1.

# CONTOH: *BINARY SEARCH*

- Terlihat bahwa usaha untuk membagi input menjadi subproblem adalah konstan (tidak ada loop). Dengan demikian, f(n) = 1.

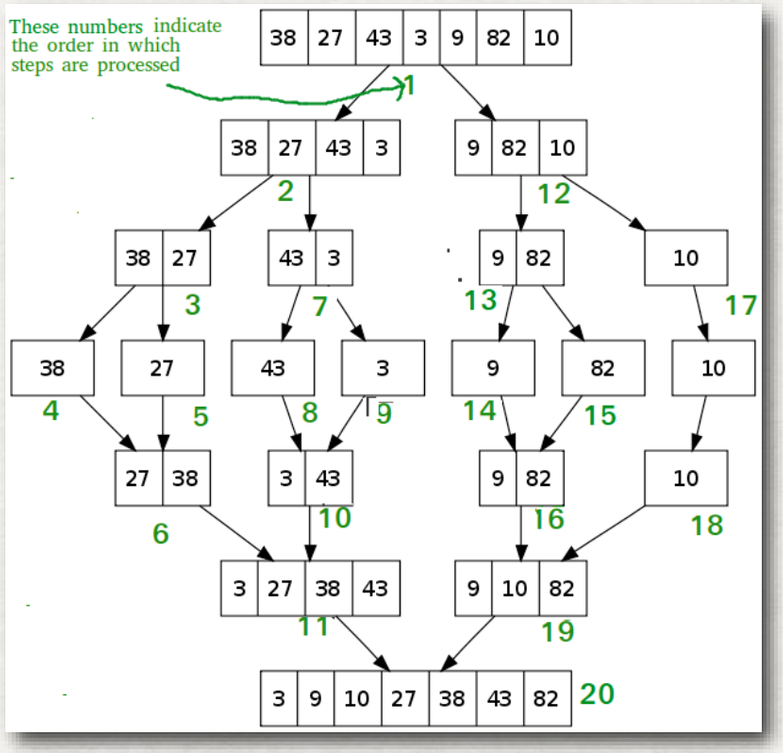
```
1   int binarySearch(int arr[], int l, int r, int x)
2   {
3       if (r >= l) {
4           int mid = l + (r - l) / 2;
5           if (arr[mid] == x)
6               return mid;
7           if (arr[mid] > x)
8               return binarySearch(arr, l, mid - 1, x);
9           return binarySearch(arr, mid + 1, r, x);
10      }
11      return -1;
12  }
```

- Jadi, relasi rekurensi dari binary search adalah T(n) = T(n/2) + 1.
- Perhatikan bahwa kompleksitasnya belum kita hitung.

# CONTOH: *MERGE SORT*

- Terlihat bahwa pada setiap langkah rekursif, merge sort akan membagi masukan menjadi dua subproblem sama besar. Setiap bagian kemudian diurutkan menggunakan merge sort.

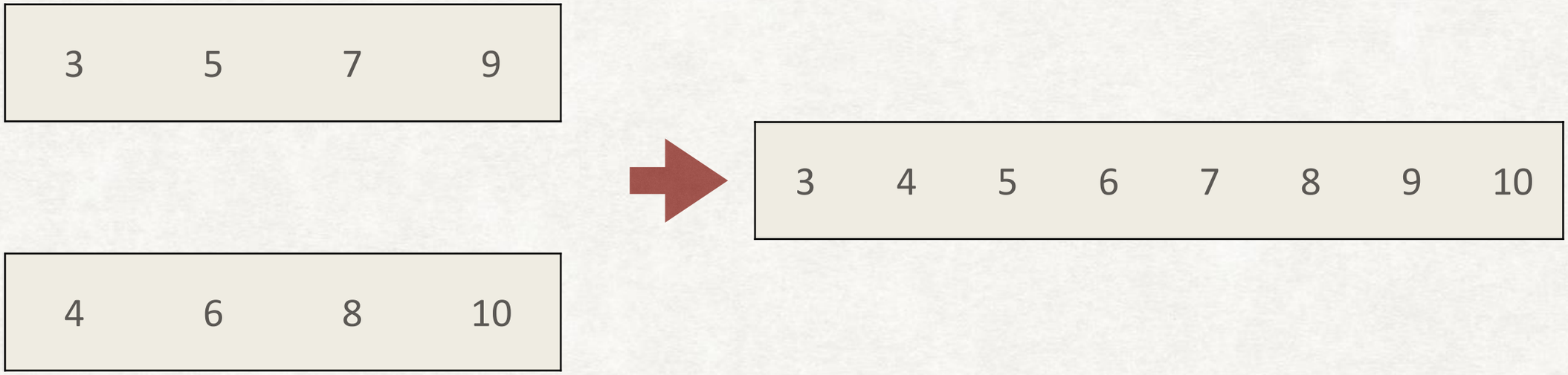
```
MergeSort(arr[], l,  r)
If r > l
     1. Find the middle point to divide the array into two halves:
             middle m = (l+r)/2
     2. Call mergeSort for first half:
             Call mergeSort(arr, l, m)
     3. Call mergeSort for second half:
             Call mergeSort(arr, m+1, r)
     4. Merge the two halves sorted in step 2 and 3:
             Call merge(arr, l, m, r)
```




- Terlihat jelas bahwa a = 2 dan b = 2.
- Bagaimana dengan f(n)? Kita harus lihat bagaimana fungsi merge bekerja.

# CONTOH: *MERGE SORT*

- Pada fungsi merge, dua subproblem berukuran n/2 yang terurut akan disatukan menjadi sebuah list berukuran n.
- Perbandingan maksimal yang dilakukan adalah n-1.




- Fungsi ini memiliki kompleksitas O(n).
- Dengan demikian, relasi rekurensi dari merge sort adalah T(n) = 2T(n/2) + n.

# MENYELESAIKAN RELASI REKURENSI

- Ada tiga cara populer yang dapat digunakan untuk menyelesaikan relasi rekurensi untuk mendapatkan kompleksitas dari suatu algoritme rekursif.
  - Teknik Subtitusi (guess and check).
  - Teknik Pohon Rekursif.
  - Teknik Teorema Master.
- Pada bagian selanjutnya, kita akan lihat ketiga cara tersebut.

# TEKNIK SUBTITUSI

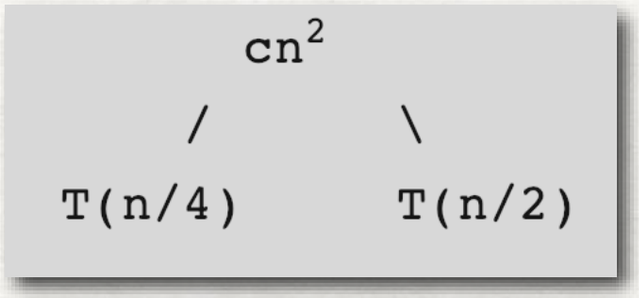
- Kita tebak sebuah nilai big-Oh dari suatu relasi rekurensi dan membuktikannya dengan menggunakan induksi (guess and check).
- Berguna untuk kasus non-trivial yang tidak bisa diselesaikan oleh teorema master.

# TEKNIK POHON REKURSIF

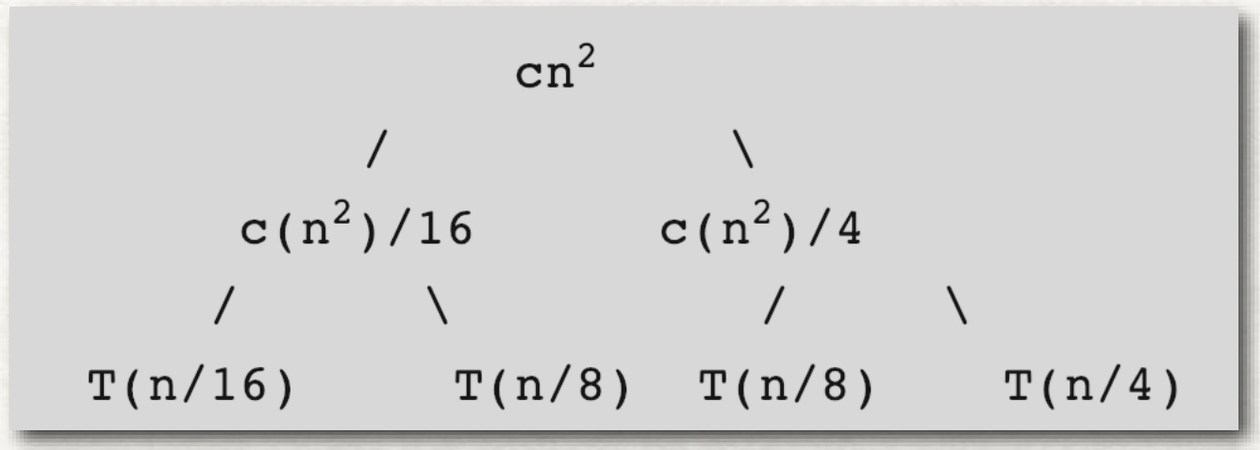
- Menggambar pohon rekursif (hasil trace dari algoritme rekursif).
- Jumlahkan waktu pada setiap level pada tree.
- Mencari pola dari deret yang ditemukan untuk menghitung jumlah pekerjaan dari keseluruhan tree.
- Mirip dengan menebak rumus umum dari suatu deret yang diberikan.

# CONTOH

- Diberikan relasi $T(n) = T(n/4) + T(n/2) + cn^2$
- Jika digambarkan dalam bentuk pohon:



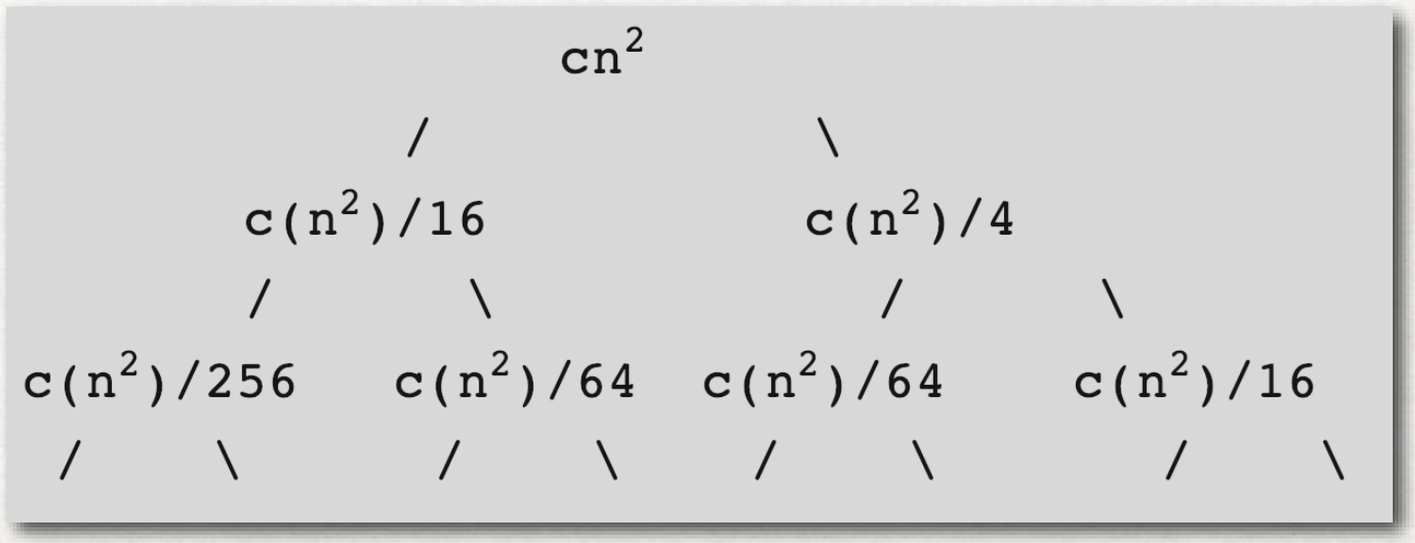

- Lanjutkan dengan penjabaran T(n/4) dan T(n/2):




- Lanjutkan untuk satu level berikutnya!

# CONTOH

- Sekarang, jumlahkan cost dari setiap level, dan cari pola.

| Level | Tree | Cost |
|---|---|---|
| 0 |  | $cn^2$ |
| 1 | | $(5/16)cn^2$ |
| 2 | | $(25/256)cn^2$ |



- Terlihat bahwa cost pada level ke-i mengikuti rumus $(5/16)^i cn^2$
- Barisan geometri dengan rasio 5/16.

# CONTOH

- Berikutnya hitung jumlah barisan tersebut jika menuju ke tak hingga: Sn = $cn^2 + (5/16)^1 cn^2 + (5/16)^2 cn^2 + (5/16)^3 cn^2 + ….$
- Sn = $cn^2 / (1 - 5/16) = (16/11) cn^2 = O(n^2)$
- Dengan demikian, $T(n) = T(n/4) + T(n/2) + cn^2 = O(n^2)$

# TEKNIK TEOREMA MASTER

- Metode master merupakan cara singkat untuk menyelesaikan relasi rekurensi dengan bentuk umum:

$$T(n) = aT(n/b) + f(n)$$

Jumlah subproblem.

Ukuran dari subproblem relatif terhadap ukuran masalah asli.

Waktu yang diperlukan untuk membagi subproblem dan menyatukannya kembali.

- dengan a ≥ 1 dan b > 1 adalah konstanta dan f(n) adalah fungsi yang secara asimtotik positif.

# TEKNIK TEOREMA MASTER

- Terdapat tiga kasus umum pada teorema master:

| | **Kondisi** | **Kompleksitas** |
|---|---|---|
| Kasus 1 | jika f($n$) = O($n^{\log_b a}$) | T($n$) = Θ($n^{\log_b a}$) |
| Kasus 2 | jika f($n$) = Θ($n^{\log_b a}$) | T($n$) = Θ($n^{\log_b a}$ log $n$) |
| Kasus 3 | jika f($n$) = Ω($n^{\log_b a}$) | T($n$) = Θ(f($n$)) |

- Apabila syarat a ≥ 1 dan b > 1 adalah konstanta dan f(n) adalah fungsi yang secara asimtotik positif tidak dipenuhi, berarti teorema master tidak dapat digunakan.

# CONTOH

Pertanyaan:

Berapakah solusi dari relasi rekurensi $T(n) = 4T(n/2) + n^2$?

Jawaban:

$a = 4$, $b = 2$, $f(n) = n^2$ —> Syarat dipenuhi

Karena $f(n) = n^2 =$ , gunakan kasus II teorema master.

Jadi, $T(n) = \Theta(n^2 \log n)$

# LATIHAN TEOREMA MASTER

- Selesaikan relasi rekurens berikut menggunakan teknik teorema master!

| No | Relasi Rekurensi | a | b | f(n) | $n^{\log_b a}$ | f(n) =...( $n^{\log_a b}$ )? | Kasus | Hasil |
|---|---|---|---|---|---|---|---|---|
| 1 | $T(n) = 3T(n/2) + n^2$ | | | | | | | |
| 2 | $T(n) = 4T(n/2) + n^2$ | | | | | | | |
| 3 | $T(n) = T(n/2) + 2^n$ | | | | | | | |
| 4 | $T(n) = 2^n T(n/2) + n^n$ | | | | | | | |
| 5 | $T(n) = 16T(n/4) + n$ | | | | | | | |

# LATIHAN TEOREMA MASTER

- Jawaban

| No | Relasi Rekurensi | a | b | f(n) | $n^{\log_b a}$ | f(n) =...( $n^{\log_a b}$ )? | Kasus | Hasil |
|---|---|---|---|---|---|---|---|---|
| 1 | T($n$) = 3T ($n$/2) + $n^2$ | 3 | 2 | $n^2$ | $n^{\log_2 3}$ | Ω | 3 | Θ($n^2$) |
| 2 | T($n$) = 4T($n$/2) + $n^2$ | 4 | 2 | $n^2$ | $n^2$ | Θ | 2 | Θ($n^2 \log n$) |
| 3 | T($n$) = T($n$/2) + $2^n$ | 1 | 2 | $2^n$ | 1 | Ω | 3 | Θ($2^n$) |
| 4 | T($n$) = $2^n$T($n$/2) + $n^n$ | $2^n$ | 2 | $n^n$ | - | - | - | - |
| 5 | T($n$) = 16T($n$/4) + $n$ | 16 | 4 | $n$ | $n^2$ | O | 1 | Θ($n^2$) |

# PENUTUP

- Perhatikan bahwa relasi rekurensi yang diberikan pada kuliah ini mengasumsikan ukuran setiap subproblem sama.

$$T(n) = aT(n/b) + f(n)$$

Jumlah subproblem.

Ukuran dari subproblem relatif terhadap ukuran masalah asli.

Waktu yang diperlukan untuk membagi subproblem dan menyatukannya kembali.

- Mungkinkah ada algoritme yang ukuran subproblemnya berbeda? Tentu iya.

# PENUTUP

- Metode Akra-Bazzi merupakan generalisasi dari teorema master yang memungkinkan penyelesaian relasi rekurensi yang lebih luas dengan bentuk umum:

$$T(x) = g(x) + \sum_{i=1}^{k} a_i T(b_i x + h_i(x)) \qquad \text{for } x \geq x_0.$$

- Dengan kata lain, teknik ini bisa menghitung kompleksitas algoritme yang membagi ke k jenis subproblem dengan ukuran a dan b yang berbeda dan fungsi merge yang berbeda.

- Tidak dibahas lebih lanjut di mata kuliah ini.

# PERTEMUAN SELANJUTNYA (6-7)

- Studi kasus perancangan dan analisis algoritme iteratif dan rekursif dengan memanfaatkan teknik-teknik yang telah diberikan.
- Beberapa kasus di Algoritme, Dasar Pemrograman, dan Struktur Data akan dimunculkan kembali.